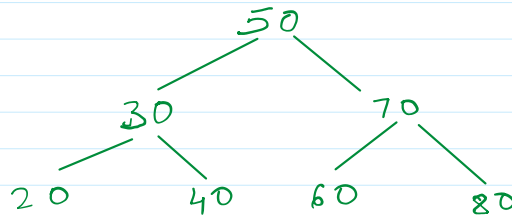


Binary Search Tree (BST)

11 June 2026 10:06

A **Binary Search Tree (BST)** is a special type of tree in which:

- Every node has at most two children (0, 1 or 2)
- The left subtree contains values smaller than the parent node.
- The right subtree contains values greater or equal to the parent node.
- Both left and right subtrees are themselves BST.



BST property: For every node.
Left child < Parent ≤ Right child.

Why use binary search trees?

BST provides efficient operations for:

- Insertion
- Searching
- Deletion



'n' ← we call it linear
↑
 $O(n)$

Disadvantage →
(insertion of
one element takes
linear time).

↑ to make this cell empty, I need
to shift 'n' elements.
↑ Shifting is costly in Array.
↑ why?
we need to shift 'n' elements.

Advantage of array: If you know the index, you can access the
element directly.

n^2 ← quadratic → $O(n^2)$
 n^3 ← cubic → $O(n^3)$

→ I want constant time for insertion. $O(1)$
↑ constant.

Linked List.

→ Insertion
→ Deletion } Shifting of elements
not needed.

↑ In Linked list, search is inefficient.

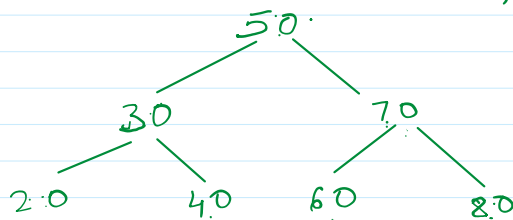
Disadvantage → is linear

Disadvantage $\rightarrow$ is linear $\rightarrow$ ∞
 To make our search efficient (BST).

Insertion
 Deletion
 and search } are efficient. (It uses binary search logic).

BST always maintains order.

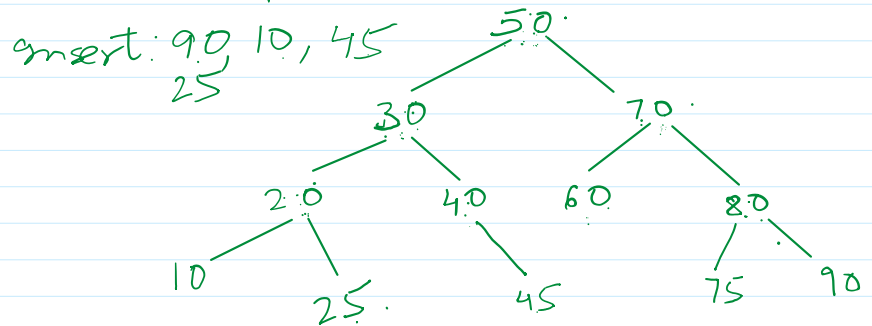
In-order traversal of BST you will get sorted data or data in order.



In order $\rightarrow L \rightarrow R \rightarrow R = 20, 30, 40, 50, 60, 70, 80$

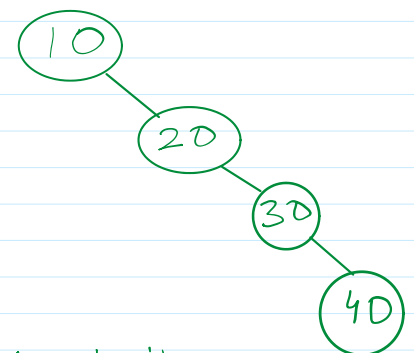
Time complexity (Average Case)

Operation	Complexity
insert	$O(\log n)$
delete	$O(\log n)$
search	$O(\log n)$



```

package BinaryTree;
public class Node {
    private int data;
    private Node left, right;
    public Node(){
        data = 0;
        left = right = null;
    }
    public Node(int data) {
        this.data = data;
    }
    public int getData() {
        return data;
    }
    public void setData(int data) {
        this.data = data;
    }
    public Node getLeft() {
        return left;
    }
    public void setLeft(Node left) {
        this.left = left;
    }
  
```



Complexity becomes: $O(n)$.
 $\uparrow$
 [A.V.L Tree]

```
}  
public Node getRight() {  
    return right;  
}  
public void setRight(Node right) {  
    this.right = right;  
}  
@Override  
public String toString() {  
    return "Node [data=" + data + " ]";  
}  
}
```

BST Operations

Insertion:

1. If tree is empty -> create root
2. If value is smaller -> move left.
3. If value is larger -> move right.
4. Repeat until NULL is found.